

▼ **Typecasting:**

Typecasting in Python, also known as type conversion, refers to converting one data type into another. Python provides several built-in functions to facilitate typecasting.

▼ **Types of Typecasting:**

1. Implicit Typecasting
2. Explicit Typecasting

complex

↑

float

↑

int

↑

bool

Unlike some languages, Python **does not automatically convert numbers to strings**

python

```
a = "Hello"
b = 5
# print(a + b)    ✗ TypeError
print(a + str(b))  # ✓ "Hello5"
```

2. Explicit Typecasting:

Explicit typecasting is performed when the programmer explicitly converts one data type to another using built-in functions. This is also known as type conversion.

Common Typecasting Functions

- `int()`: Converts a value to an integer.
- `float()`: Converts a value to a float.
- `str()`: Converts a value to a string.
- `list()`: Converts a value to a list.
- `tuple()`: Converts a value to a tuple.
- `set()`: Converts a value to a set.
- `dict()`: Converts a value to a dictionary (from a list of key-value pairs).
- `bool()`: Converts a value to a boolean.

✓ Slicing:

Slicing is a powerful feature in Python that allows you to extract specific parts of sequences such as strings, lists, or tuples by specifying a range of indices. The basic syntax for slicing is:

Syntax: object[start:stop:step]

[+ Code](#)[+ Text](#)

A diagram showing a string 'SLICET' with its characters each in a purple box. Above the boxes are indices: -7 for 'S', -6 for 'L', -5 for 'I', -4 for 'C', -3 for 'E', -2 for 'I', -1 for 'T', and 0 for the end of the string. To the left of the first box is 'L ='.

A diagram showing a slice operation. 'L' followed by '[-5:-2]' in orange, followed by '=', then three orange boxes containing 'I', 'C', and 'E'. Below the first box is '-5' and the text 'Start of the Slice'. Below the last box is '-2' and the text 'End of the Slice'.

- start: The starting index of the slice. The element at this index is included in the slice.
- stop: The ending index of the slice. The element at this index is not included in the slice.
- step: The interval between elements in the slice. This parameter is optional and defaults to 1.

✓ **Control Flow:**

✓ 1. if-else:

The if-else statement in Python is a control statement used for decision making.

- The if block runs when the given condition is True.
- If the condition is False, the else block runs instead.
- It ensures that only one block executes depending on the condition.
- It is often combined with elif for checking multiple conditions.

SYNTAX:

if :

IF BLOCK executes if the condition is true

else:

ELSE BLOCK executes if the condition is false

✓ 2. Loops:

Loops are tools that allow you to repeat a block of code multiple times. They are helpful when you need to perform the same action or set of actions many times, such as iterating over items in a list or repeating an operation until a certain condition is met.

✓ Types of Loops:

A. For Loop:

A for loop is a type of loop that is used to iterate over a sequence of values. In Python, a sequence can be any iterable object, such as a list, tuple, or string. The basic syntax for a for loop in Python is as follows:

for variable in iterable: # code block # (runs once for each element in iterable) else: # optional block # (runs if loop completes normally, without 'break')

B. while loop

A while loop is another type of loop in Python. Unlike a for loop, which iterates over a sequence of values, a while loop repeatedly executes a block of code as long as a certain condition is true. The basic syntax for a while loop in Python is as follows:

```
while condition: # code block # (runs as long as condition is True) else: # optional block # (runs only if loop ends normally, without 'break')
```

✓ break statement:

- The break statement is used to terminate the loop immediately.
- Control moves to the first statement after the loop.
- Useful when you want to stop looping once a condition is met

Example: Write a Python program that uses a for loop and a break statement to stop the loop when the counter reaches 5, and also prints a message after each iteration (except when the loop breaks). Finally, print a message outside the loop.

✓ continue statement:

Skips an iteration. Works with both for loops and while loops.

Example Question:

- Write a program that continuously asks the user to provide an input number.
- Print the square of that input number.
- The program should stop if the user enters 5.
- In case user enters any multiple of 4, skip it.

1. Lists

A list is a versatile data structure that allows you to store a collection of items in a single variable. Lists can hold items of different types and are mutable, meaning their contents can be changed after creation. Here's a comprehensive guide to Python lists:

List Operation	Description	Example	Output
append()	Used to add an element to the end of the list.	<pre>my_list = [1, 2, 3] my_list.append(4)</pre>	<pre>[1, 2, 3, 4]</pre>
pop()	Used to remove and return the last element of the list.	<pre>my_list = [1, 2, 3] last_element = my_list.pop()</pre>	<pre>last_element = 3</pre>
len()	Used to get the length of the list.	<pre>my_list = [1, 2, 3] length = len(my_list)</pre>	<pre>length = 3</pre>
sort()	Used to sort the list in ascending order.	<pre>my_list = [3, 1, 2] my_list.sort()</pre>	<pre>[1, 2, 3]</pre>

1. Case Conversion

- `upper()` → Converts all characters to uppercase.
- `lower()` → Converts all characters to lowercase.
- `title()` → Capitalizes the first letter of each word.
- `capitalize()` → Capitalizes the first character of the string.
- `swapcase()` → Swaps uppercase to lowercase and vice versa.

2. Searching

- `find(sub)` → Returns the index of the first occurrence of substring, or -1 if not found.
- `rfind(sub)` → Returns the index of the last occurrence of substring.
- `index(sub)` → Same as `find()`, but raises an error if substring not found.
- `count(sub)` → Returns the number of times a substring occurs.

3. Validation / Checking

- `isalpha()` → Checks if all characters are alphabets.
- `isdigit()` → Checks if all characters are digits.
- `isalnum()` → Checks if all characters are alphanumeric (letters + digits).
- `isspace()` → Checks if string contains only whitespace.
- `isupper()` → Checks if all characters are uppercase.
- `islower()` → Checks if all characters are lowercase.

4. Modification / Trimming

- `strip()` → Removes whitespace (or given characters) from both ends.
- `lstrip()` → Removes whitespace from the left side.
- `rstrip()` → Removes whitespace from the right side.
- `replace(old, new)` → Replaces all occurrences of a substring with another.

5. Splitting & Joining

- `split(sep)` → Splits string into a list using the given separator.
- `join(iterable)` → Joins elements of an iterable into a single string.

6. Alignment & Padding

- `center(width, fillchar)` → Centers string in a given width with optional fill character.
- `ljust(width, fillchar)` → Left-aligns string within given width.
- `rjust(width, fillchar)` → Right-aligns string within given width.
- `zfill(width)` → Pads string on the left with zeros to reach specified width.

7. String Formatting

- `format()` → Inserts values into placeholders `{}` in a string.
- **f-strings** → (Python 3.6+) Embed variables/expressions directly inside a string using `{}`.

Similarly, the following kinds of functions can be used:

- `cmp(tuple1, tuple2)`

Compares elements of two different tuples.

- `len(tuple)`

Returns the length of the tuple.

- `max(tuple)`

Returns the element with the maximum value from the tuple.

- `min(tuple)`

Returns the element with the minimum value from the tuple.

- `tuple(seq)`

Returns a tuple by converting a sequence (like a list) to a tuple.

Category	Tuple (Immutable)	Set (Mutable, Unique Elements)	Dictionary (Key-Value Pairs)
Creation	<code>tuple(iterable)</code>	<code>set(iterable)</code>	<code>dict()</code> / <code>{}</code>
Basic Ops	<code>len()</code> , <code>max()</code> , <code>min()</code> , <code>sum()</code> , <code>any()</code> , <code>all()</code> , <code>sorted()</code>	Same built-ins work: <code>len()</code> , <code>max()</code> , <code>min()</code> , <code>sum()</code> , <code>any()</code> , <code>all()</code> , <code>sorted()</code>	<code>len()</code> , <code>sorted()</code> , <code>any()</code> , <code>all()</code> (on keys)
Methods	<code>.count(x)</code> → count element <code>.index(x)</code> → first index	<code>.add(x)</code> <code>.remove(x)</code> <code>.discard(x)</code> <code>.pop()</code> <code>.clear()</code> <code>.copy()</code>	<code>.get(key, default)</code> <code>.keys()</code> <code>.values()</code> <code>.items()</code> <code>.update(other)</code> <code>.copy()</code> <code>.setdefault(key, default)</code> <code>.fromkeys(seq, value)</code> <code>.pop(key[, default])</code> <code>.popitem()</code> <code>.clear()</code>
Set/Map Ops	✗	<code>.union()</code> / <code>`</code>	<code>
.intersection() / &
.difference() / -
.symmetric_difference() / ^
r>.issubset() / <=
.issuperset() / >=
.isdisjoint() `</code>
Mutability	✗ Immutable	✓ Mutable	✓ Mutable
Duplicates	✓ Allowed	✗ Not allowed	Keys ✗, Values ✓
Indexing	✓ Supports indexing/slicing	✗ Not ordered (before Python 3.7)	✓ Ordered from Python 3.7+ (keys)

- Functional programming (FP) is a way of writing programs where you use functions to get things done, like solving math problems.
- It's different from the usual way of programming, which focuses on telling the computer how to do things step by step.

✓ What is Functional Programming?

- Functional Programming (FP) is a style of programming where you use functions, just like in math, to perform tasks.
- It avoids changing data and keeps things simple by not allowing data to be modified once it is created.
- Functional programming (FP) is a programming paradigm that treats computation as the evaluation of mathematical functions and avoids changing state and mutable data.
- Unlike imperative programming, where the focus is on how to perform tasks (using loops, conditionals, and state changes), functional programming emphasizes what to do, using expressions that result in values.

- **Functional Programming (FP):**

- Focuses on computation through mathematical functions.
- Emphasizes *what* to do rather than *how* to do it.
- Relies on pure functions, immutability, and function composition.

- **Imperative Programming (IP):**

- Focuses on *how* to perform tasks using a sequence of instructions that change program state.
- Emphasizes step-by-step procedures and control flow, such as loops and conditionals.
- Typical languages: C, C++, Python (in imperative style), JavaScript (in imperative style).

✓ Function

- Functions in Python are blocks of reusable code that perform a specific task.
- They help in organizing code, making it more modular, readable, and maintainable.

✓ 1. Defining a Function

- To define a function in Python, you use the `def` keyword followed by the function name, parentheses (which may contain parameters), and a colon.
- The function body is indented and contains the code that the function will execute.

[]

```
def function_name(parameters):  
    # Function body  
    return value
```

- **function_name**: The name of the function.
- **parameters**: Optional. Variables passed to the function for processing.
- **return**: Optional. Specifies the value that the function returns.

Topics to cover

Functions

Calling function

Parameters and arguments - Position, keyword, default, arbitrary, arbitrary keyword argument, returning value

Pure and impure functions

Lambda functions

Map

Filter

Map and filter using Lambda

Zip and unzip

✓ **Pure Functions**

A pure function is a fundamental concept in functional programming that adheres to specific criteria, making it predictable, testable, and reliable.

✓ **1. Definition of Pure Functions**

- **Deterministic Output:**

- A pure function always produces the same output for the same set of inputs, no matter when or where it is called. This is often referred to as determinism.

- **No Side Effects:**

- A pure function does not cause any observable side effects outside of returning a value.
- This means it doesn't modify any external state, such as variables, files, or data structures, and it doesn't perform operations like logging, printing, or writing to a database.

Normal function

```
def add(x, y):  
    return x + y
```

Same thing using lambda

```
add_lambda = lambda x, y: x + y
```

```
print(add_lambda(5, 3))
```

```
numbers = [1, 2, 3, 4, 5]
```

```
squared = list(map(lambda x: x ** 2, numbers))
```

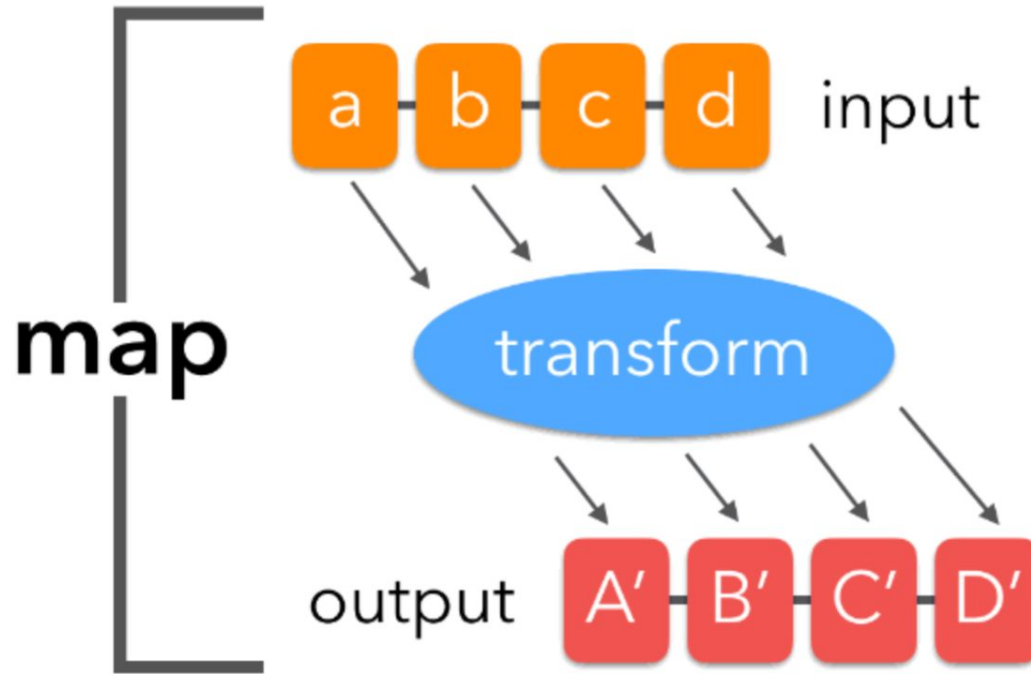
```
print(squared)
```

```
numbers = [1, 2, 3, 4, 5, 6]
```

```
evens = list(filter(lambda x: x % 2 == 0, numbers))
```

```
print(evens)
```


✓ MAPS



- The map function applies a given function to each item in an iterable (e.g., a list) and returns a new iterable with the results.
- It's an efficient way to transform data, applying the same operation to multiple elements.

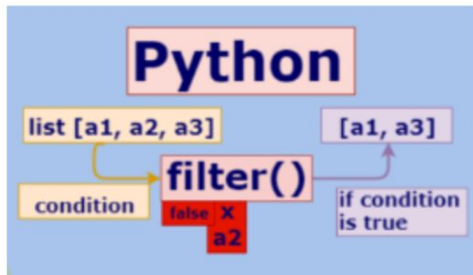


Basic Syntax

```
map(function, iterable, ...)
```

- **function:** The function to apply to each element of the iterable.
- **iterable:** One or more iterable(s) (like lists, tuples, etc.) to which the function will be applied.

✓ FILTERS



The filter function filters elements from an iterable based on a specified condition. It's useful for selecting specific data points that meet certain criteria.

✓ Basic Syntax

```
filter(function, iterable)
```

- **function:** A function that tests each element of the iterable. It should return `True` to include the element in the result and `False` otherwise.
- **iterable:** The iterable (like a list, tuple, or string) whose elements you want to filter.

The zip function in Python is a versatile tool for combining multiple iterables into a single iterable, creating pairs or tuples from corresponding elements of the input sequences. It takes two or more sequences as input and returns an iterator that generates tuples, where each tuple contains elements from the input sequences at the same index.

```
names = ['Simran', 'Aman', 'Riya']  
scores = [90, 85, 88]
```

```
zipped = zip(names, scores)  
print(list(zipped))
```

Basic Syntax

```
zip(*iterables)
```

✓ unzip

unzip is not a built-in function in Python, but it refers to the process of separating a zipped object back into individual iterables. This can be done using the `zip()` function with the unpacking operator `(*)`.

```
zipped_data = [('Simran', 90),  
('Aman', 85), ('Riya', 88)]  
names, scores = zip(*zipped_data)  
print(names)  
print(scores)
```

REDUCE

reduce function in Python



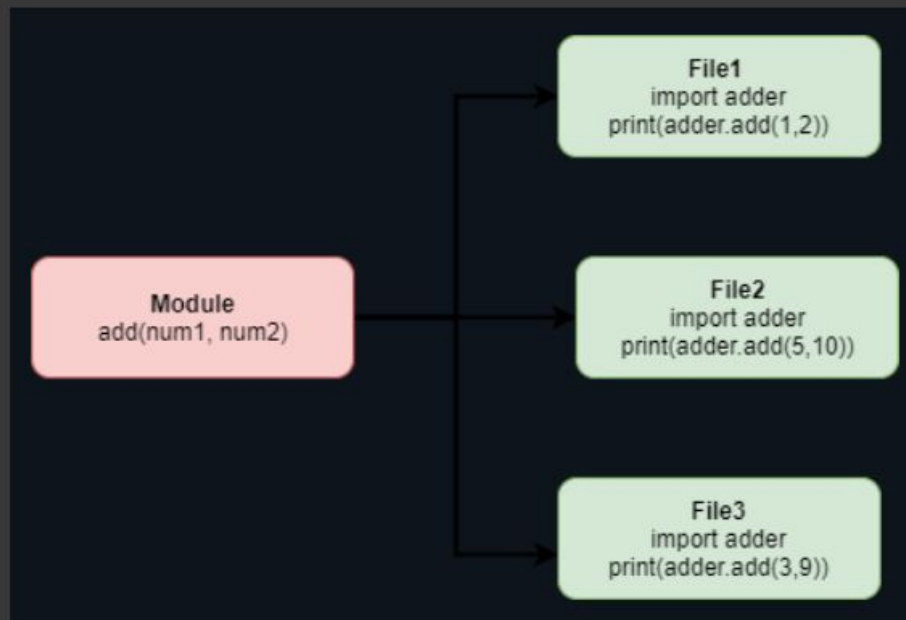
The reduce function (from the functools module) aggregates elements in an iterable using a specified function. It's often used for cumulative operations, like summing all elements in a list.

```
from functools import reduce  
  
reduce(function, iterable[, initializer])
```



✓ What is Python Module?

- A document with definitions of functions and various statements written in Python is called a Python module.
- It encapsulates functions, classes, and variables.
- Standard library modules provide built-in functionalities (e.g., math, os, random).



Topics for today

- Modules
- Packages
- Namespace
- Exception Handling
- Custom Exception
- Else block
- Finally
- Base exception
- Raising Exception
-

✓ Why modules are useful:-

- Modules provide a structured and scalable way to manage and organize code in large projects. They facilitate reusability, maintainability, and collaboration, making the development process more efficient and the codebase more robust.
- Reusability: Once a module is created, it can be reused across different parts of the project or even in other projects. This reduces duplication of code and promotes the DRY (Don't Repeat Yourself) principle.
- Scalability: As projects grow, maintaining a monolithic codebase becomes challenging. By using modules, you can scale your project more effectively, as changes to one module won't necessarily affect others.
- Namespace Management: Modules create separate namespaces, which help prevent name collisions in large projects. This means you can have functions, variables, or classes with the same name in different modules without conflict.
- Dependency Management: Modules allow you to clearly define dependencies between different parts of the code. This makes it easier to manage and update dependencies without affecting unrelated parts of the project.

◆ What is a package in Python?

A **package** in Python is just a **folder** that contains **Python modules** (files) and an optional `__init__.py` file.

It's a way to **organize related modules together** so that your code is modular, reusable, and easy to manage.

◆ Components

- **Module** → a single `.py` file (e.g., `math_utils.py`)
- **Package** → a folder containing multiple modules (e.g., `math_package/`)
- **Subpackage** → a package inside another package

◆ Why packages?

- Organize big projects into smaller pieces
 - Avoid name conflicts (namespace management)
 - Reuse functionality across projects
-

✨ In short:

- **Module** = single file
- **Package** = folder of modules
- **Library** = collection of packages (like NumPy, Pandas, TensorFlow)

◆ The `__name__` variable

Every Python file (`.py`) has a special built-in variable called `__name__`.

- If the file is **run directly** → `__name__` becomes `"__main__"`.
- If the file is **imported as a module** → `__name__` becomes the module's name.



Exception Handling

What is Exception?

- An exception in Python is an incident that happens while executing a program that causes the regular course of the program's commands to be disrupted. When a Python code comes across a condition it can't handle, it raises an exception. An object in Python that describes an error is called an exception.

What is Exception Handling?

- Exception handling in Python allows you to manage and respond to runtime errors gracefully. By catching and handling exceptions, you can prevent your program from crashing and provide meaningful error messages or recovery options.
- Exceptions are raised when some internal events occur which change the normal flow of the program.

Why Exception handling is important?

- Exception Handling is essential for managing errors gracefully and preventing program crashes.
- It improves the robustness, readability, and maintainability of code by separating error-handling logic from regular code.
- It facilitates resource management, custom error responses, and controlled flow, leading to better user experiences and more reliable applications.

try:



Run this code

except:



Execute this code when
there is an exception

else:



No exceptions? Run this
code.

finally:



Always run this code.

Introduction to OOPs:

- Object-Oriented Programming (OOP) is a programming paradigm based on the concept of "objects". It allows you to model real-world entities and their interactions in a structured and modular way.
- The main concept of object-oriented Programming (OOPs) or oops concepts in Python is to bind the data and the functions that work together as a single unit so that no other part of the code can access this data.

Why OOP?

- OOP enhances code organization, reusability, flexibility, and maintainability, making it a valuable approach for developing complex and scalable software systems.
- Example: Imagine building a car simulation. Using OOP, you can create a Car class with attributes like color, model, and speed, and methods like start(), stop(), and accelerate(). This approach is more organized and efficient than procedural programming, where you might have a bunch of unrelated functions to handle car-related operations.

✓ Core Concepts of OOPs:



✓ 1. Classes and Objects:

✓ Classes:

- A class in Python is a blueprint or template for creating objects. It encapsulates data for the object and methods to manipulate that data. Classes are a way to bundle data and functionality together.
- Key Points:
 - A class can contain attributes (variables) and methods (functions).
 - It defines the properties and behaviors of the objects that are created from it.

Objects:

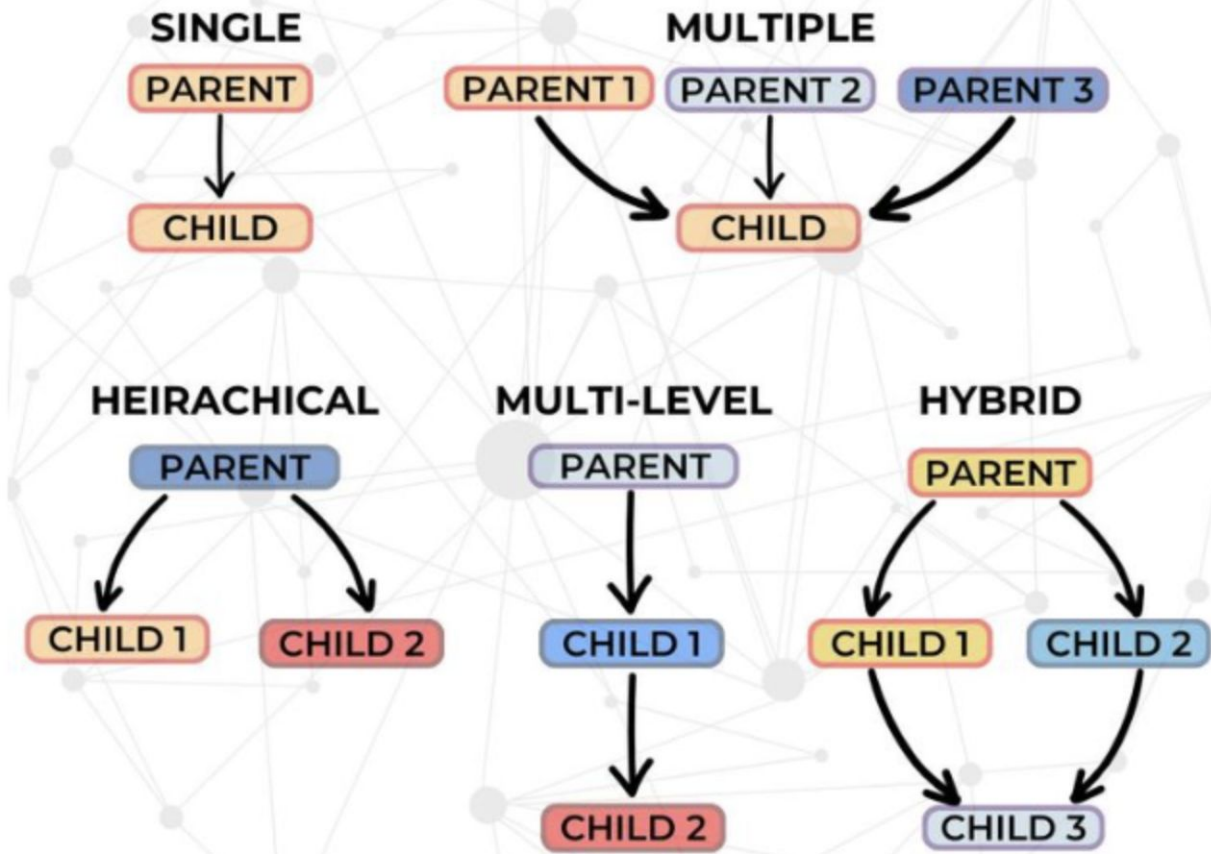
- An object is an instance of a class. When a class is defined, no memory is allocated until an object of that class is instantiated. Each object can have its own attributes and methods, defined by the class.
- Key Points:
 - Objects are individual instances of a class.
 - Each object can hold its own unique data and behavior, as defined by the class.

✓ 3. Inheritance



- Inheritance allows a new class (child or subclass) to inherit properties and behaviors (attributes and methods) from an existing class (parent or superclass). This mechanism enables the creation of a more specialized class that builds upon the general functionality of the parent class.
- Key Concepts:
 1. Parent Class (Superclass): The class whose attributes and methods are inherited.
 2. Child Class (Subclass): The class that inherits from the parent class.
- Types of Inheritance:-
 - Single Inheritance
 - Multiple Inheritance
 - Multilevel Inheritance
 - Hierarchical Inheritance
 - Hybrid Inheritance

INHERITANCE TYPES



4. Encapsulation

1. What is Encapsulation?

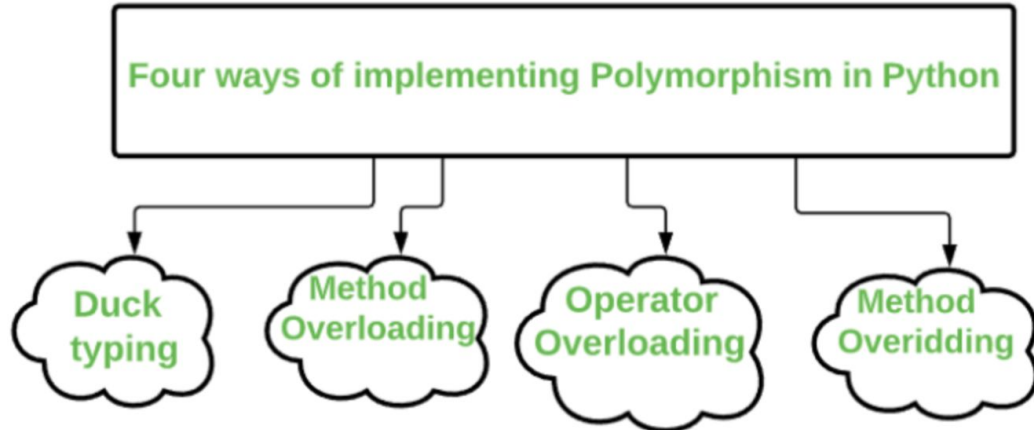
- Encapsulation is the mechanism of wrapping data (variables) and methods (functions) together as a single unit. In Python, encapsulation is achieved using classes, which act as a container for data and methods.
- Key Concepts:
 - Public Access Modifier: Allows attributes and methods to be accessed from outside the class.
 - Private Access Modifier: Restricts access to attributes and methods from outside the class.
 - Protected Access Modifier: Suggests that an attribute or method is intended for internal use by the class or its subclasses, but not from outside.

Encapsulation in Python



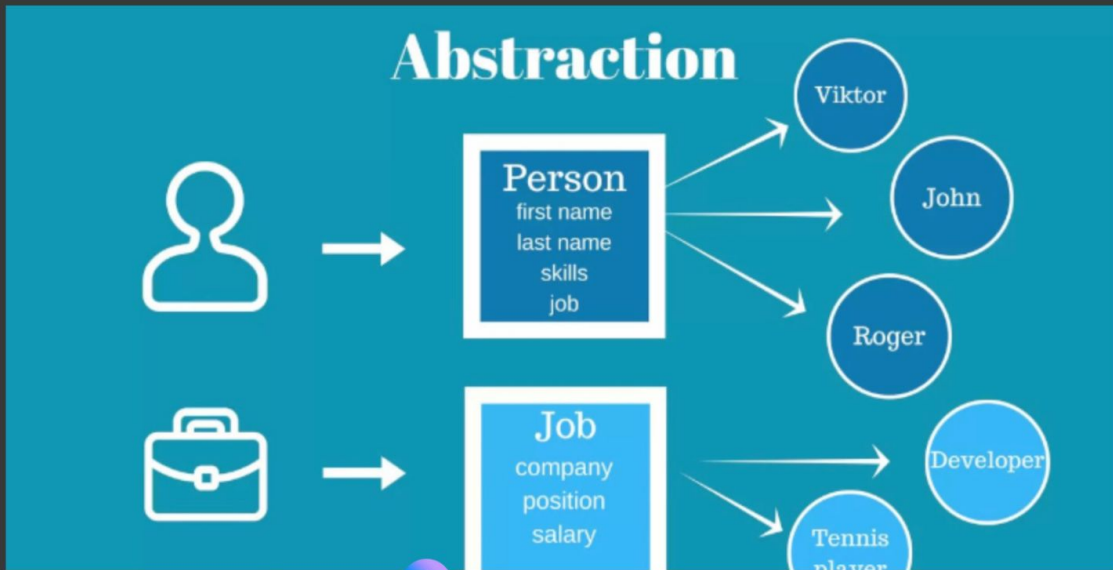
Class

- What is polymorphism?
 - Polymorphism means "many forms," and it allows the same function or method to be used on different types of objects, leading to different behaviors. The concept is used to perform a single action in different ways.
- Key Concepts:
 - Method Overriding: A subclass provides a specific implementation of a method that is already defined in its superclass.
 - Method Overloading: A single method can have multiple signatures (different parameter lists). Python doesn't support method overloading explicitly but it can be mimicked using default arguments.
 - Duck Typing: Python's dynamic typing nature where the type of an object is determined by its behavior (methods and properties), not by its inheritance from a specific class.
 - Operator Overloading: Python allows operators like `+`, `-`, `*`, etc., to be used in a polymorphic manner through operator overloading. This is done by defining special methods in the class, such as `add`, `sub`, etc.



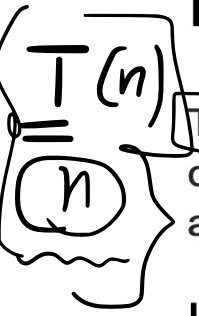
6. Data Abstraction

- What is Data Abstraction?
 - Data Abstraction is the process of exposing only the essential features of an object while hiding the implementation details. It is a way to manage complexity by presenting a simplified model of a system.
- Key Concepts:
 - Abstract Classes: Classes that cannot be instantiated and are designed to be subclassed.
 - Abstract Methods: Methods declared in an abstract class that do not have a body and must be implemented by subclasses.
 - Interfaces: In some languages, interfaces are used to define a contract for classes without specifying how they should be implemented. Python achieves similar functionality through abstract classes.

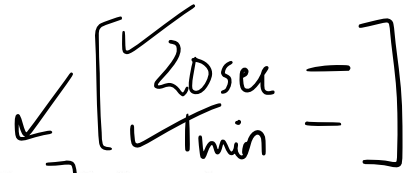


DSA - Data Structure and Algorithms

Introduction to Time Complexity:



Time complexity is a measure of the amount of time an algorithm takes to complete its execution as a function of the size of its input. It helps in assessing how the runtime of an algorithm grows with increasing input size.



Key Points:

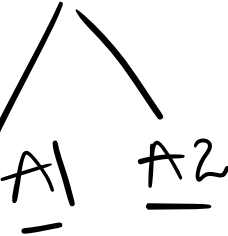
1. Asymptotic Analysis: Time complexity is often expressed using asymptotic notation (Big O, Omega, Theta) to describe the upper, lower, and tight bounds on the growth rate of an algorithm's runtime.

2. Worst-case scenario: Describes the maximum number of operations an algorithm performs for any input size.

3. Average-Case Complexity: Describes the expected number of operations averaged over all possible inputs.

4. Best-Case Complexity: Describes the minimum number of operations an algorithm performs for any input size.

5. Comparison Basis: Time complexity enables the comparison of algorithms to determine which one is more efficient for a given problem.



↓
① → ② → [Variable] → n input
✓ O(1) Constant Time Complexity:

Description: The algorithm's runtime remains constant, irrespective of the input size.

✓ Example: Accessing an element in an array by index, performing basic arithmetic operations.

✓ Python:

a[10]
1, 2, 100

repetition

x → n variable

```
def main():
```

```
→ x = 5
```

```
→ y = 10
```

```
→ result = x + y # Constant time operation
```

```
→ print("Result:", result)
```

```
if __name__ == "__main__":
```

z, w, ✓

x + y + z ... -100

$O(\log n)$ Logarithmic Time Complexity:

Description: The algorithm's runtime grows logarithmically with the input size.

Example: Binary search, certain tree operations like finding the height of a balanced binary search tree.

Python

```
def main():  
    → arr = [1, 3, 5, 7, 9, 11]  
    → target = 7  
    low, high = 0, len(arr) - 1  
    while low <= high:  
        mid = low + (high - low) // 2  
        ✓ if arr[mid] == target:  
            print("Index of", target, ":", mid) #  
Logarithmic time operation
```

```
        break
    elif arr[mid] < target:
        low = mid + 1
    else:
        high = mid - 1

if __name__ == "__main__":
    main()
```

For a recursive algorithm like binary search, we can express its running time using a recurrence relation, $T(n)$. This defines the time taken for a problem of size n in terms of the time taken for smaller subproblems. 

1. **Base Case:** If the array has only one element ($n = 1$), we perform a constant number of operations (one comparison) to determine if it's the target. Thus, $T(1) = c$, where c is a constant.

2. **Recursive Case:** For an array of size $n > 1$, the algorithm performs a constant amount of work (calculating the middle index, comparing the target to the middle element) and then makes a recursive call on one half of the array, which is a subproblem of size $n/2$. This leads to the recurrence relation:

$T(n) = T(n/2) + k$, where k is a constant representing the operations performed at each step.

3. **Solving the Recurrence:** We can solve this relation using the substitution method to find the number of times the problem size is halved until it reaches the base case.

✓ $T(n) = T(n/2) + k$

$T(n) = (T(n/4) + k) + k = T(n/2^2) + 2k$

$T(n) = (T(n/8) + k) + 2k = T(n/2^3) + 3k$

Continuing this pattern for i iterations, we get:

$T(n) = T(n/2^i) + i \cdot k$

4. **Reaching the Base Case:** The process stops when the subproblem size becomes 1. So, we set $n/2^i = 1$, which means $n = 2^i$. Solving for i gives us $i = \log_2(n)$.

5. **Final Complexity:** Substituting $i = \log_2(n)$ back into the equation:

$$T(n) = T(1) + (\log_2(n)) \cdot k$$

$T(n) = c + k \cdot \log_2(n)$ As n grows large, the constant term (c) and the constant factor (k) become insignificant. The dominant term is proportional to $\log_2(n)$. In Big O notation, this is expressed as **$O(\log n)$** .

$$n/2^k = 1 \rightarrow n = 2^k$$

$$\log_2 n = \log_2 2^k \rightarrow \log_2 n = k [\log_2 2]$$
$$\rightarrow \boxed{\log_2 n = k} = 1$$

- Initial size: n
- After 1st comparison: $n/2$
- After 2nd comparison: $n/4$
- After 3rd comparison: $n/8$
- After k comparisons:
- $n/2^k$ / 2 to the k -th power
- $n/2^k$

The algorithm terminates when the remaining search space has a size of 1.

$n/2^k = 1$ / 2 to the k -th power equals 1

$n/2^k = 1$

$n = 2^k$ equals 2 to the k -th power

$n = 2^k$

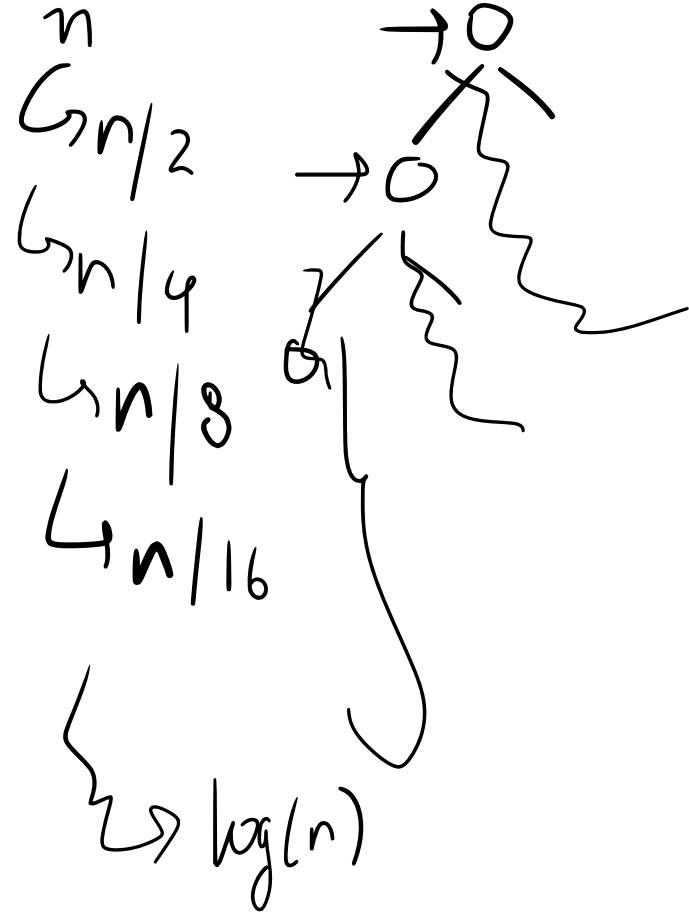
$k = \log_2(n)$ equals log base 2 of n

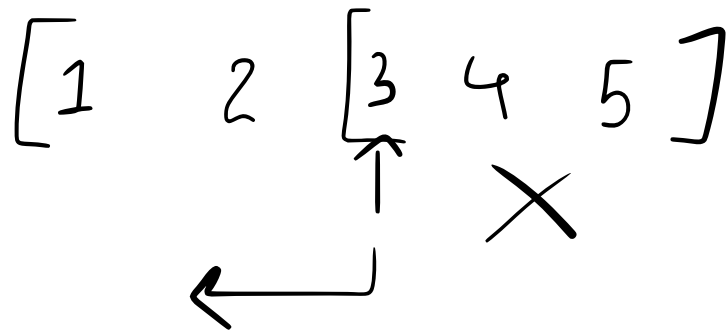
$k = \log_2(n)$

The maximum number of comparisons in the worst case is approximately

$\log_2(n)$ log base 2 of n

$\log_2(n)$ (plus one final check). This confirms the time complexity is $O(\log n)$



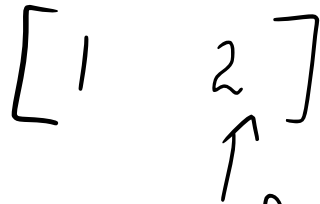


$$3 = 2$$

$$3 > 2$$

$$<$$

$k \rightarrow$



$$2 = 2$$

Binary Search

Remaining element

$$\rightarrow n$$

$$\rightarrow n/2$$

$$\rightarrow (n/2)/2 \rightarrow n/4$$

$$\rightarrow n/8 \dots \dots \dots \textcircled{1}$$

$$\boxed{\frac{n}{2^k} = 1}$$

$$\rightarrow$$

$$n = 2^k$$

$$\rightarrow$$

$$\log_2 n$$

$$= \lfloor \log_2 2 \rfloor$$

$$=$$

$$\lfloor \log n \rfloor = k$$



O(n) Linear Time Complexity:

Description: The algorithm's runtime increases linearly with the input size.

Example: Linear search, traversing through an array or list, counting the occurrences of an element in a list.

Python:

```
def main():
    arr = [1, 3, 5, 7, 9, 11]
    target = 7
    for i in range(len(arr)):
        if arr[i] == target:
            print("Index of", target, ":", i) # Linear time
            break

operation

if __name__ == "__main__":
    main()
```


$O(n^2)$ Quadratic Time Complexity:

Description: The algorithm's runtime grows quadratically with the input size.

Example: Bubble sort, selection sort, certain dynamic programming algorithms like the naive implementation of the knapsack problem.

Pseudocode

```
function hasDuplicates(array):  
    n = length(array)  
    ✓ for i from 0 to n-1:           // Outer loop iterates 'n' times  
        ✓ for j from 0 to n-1:       // Inner loop iterates 'n' times  
            if i != j and array[i] == array[j]:  
                return true  
    return false
```

$$O(n) \times O(n) \rightarrow O(n^2)$$

Derivation

1. **Outer Loop:** The first loop iterates through all n elements of the array exactly once.
2. **Inner Loop:** For each iteration of the outer loop, the second loop also iterates through all n elements of the array.
3. **Total Operations:** The number of times the comparison `array[i] == array[j]` is executed is the number of outer loop iterations multiplied by the number of inner loop iterations:
 - $n \times n = n^2$

The total number of comparisons is exactly n^2 .

4

5

6

✓ 8

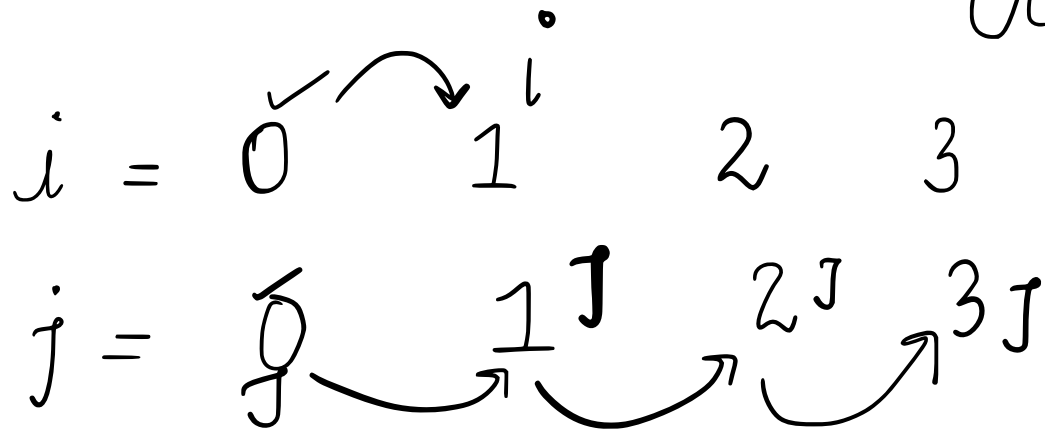
 $O(n^2)$ max = 0247

Diagram illustrating the pairs (i, j) for the dynamic programming calculation. The pairs are listed as follows:

i, j	i, j	i, j
$(4, 4)$	$(5, 5)$	$(6, 8)$
$(4, 5)$	$(5, 6)$	$(6, 6)$
$(4, 6)$	$(5, 8)$	$(8, 8)$
$(4, 8)$		

arr[i] = ~~4~~ 5arr[j] = ~~4~~ ~~5~~ ~~6~~ 8
~~5~~ 6 84, 5
5, 4

Diagram illustrating the formula for the maximum subarray sum:

$$\underbrace{|arr[i] - arr[j]|}_{2 + 3} + \underbrace{|i - j|}_{3 + 2}$$

The result is $1 + 1 = 2$ and $3 + 2 = 5$.

$O(n^k)$ Polynomial Time Complexity:



Description: The algorithm's runtime grows polynomially with the input size, where k is a constant.

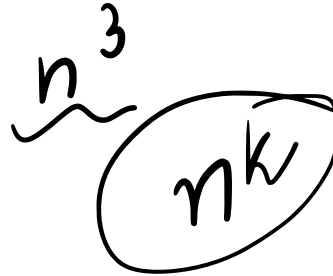
Example: Matrix multiplication using the naive approach, certain graph algorithms like the FloydWarshall algorithm.

Python

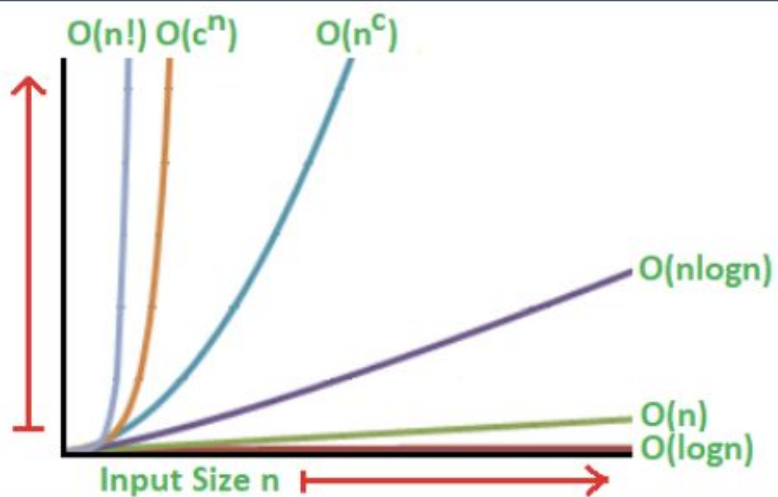
```
def main():
    A = [[1, 2], [3, 4]]
    B = [[5, 6], [7, 8]]
    result = [[0, 0], [0, 0]]

    # Polynomial time operation (Matrix Multiplication)
    for i in range(2):
        for j in range(2):
            for k in range(2):
                result[i][j] += A[i][k] * B[k][j]

if __name__ == "__main__":
    main()
```



Running Time
Complexity
in terms of
Big-O
 $O(f(n))$



$O(n!)$, $O(c^n)$, $O(n^c)$ - Worst

$O(n \log n)$ - Bad

$O(n)$ - Fair

$O(\log n)$ - Good

$O(1)$ - Best

Introduction to Space Complexity:

Space complexity is a measure of the total amount of memory an algorithm needs to run to completion as a function of the size of the input. It includes memory used by variables, data structures, function calls, and auxiliary space.

Key Concepts:

1. **Auxiliary Space:** The extra space or temporary space used by the algorithm excluding input space.
2. **Input Space:** Memory required to store the input data.
- 3. **Total Space** = Auxiliary Space + Input Space.
- 4. **Scalability:** Space complexity helps determine how well an algorithm scales with increasing input sizes.
- 5. **Trade-Offs:** Often there's a trade-off between time and space complexity.

Space Complexity of Data Structures:

Structure	Average Space Complexity
Array (size n)	$O(n)$

Graph (adjacency matrix)	$O(V^2)$
--------------------------	----------

Space Complexity in Algorithms:

Sorting Algorithms

Algorithm	Space Complexity
Bubble Sort	$O(1)$
Insertion Sort	$O(1)$

Linked List (n nodes)	$O(n)$
Stack/Queue	$O(n)$
Hash Table (n items)	$O(n)$
Binary Tree (n nodes)	$O(n)$
Graph (adjacency list)	$O(V + E)$

Merge Sort	$O(n)$
Quick Sort	$O(\log n)$ (stack)
Heap Sort	$O(1)$

Searching Algorithms

Algorithm	Space Complexity
Linear Search	$O(1)$
Binary Search	$O(\log n)$
DFS (recursive)	$O(h)$
BFS (queue)	$O(n)$